

UrbanIQ

Complete Interview Preparation Cheat Sheet

Mumbai Civic Intelligence Platform

Generated July 2026

Tech Stack at a Glance

Backend: Django 5.2 + PostGIS + DRF 3.17 + Celery 5.6

Frontend: React 19 + Vite 8 + Leaflet + Recharts 3

ML Pipeline: XGBoost 2.1 + scikit-learn + DBSCAN

Infrastructure: Docker + Render + Cloudflare Pages

External: Twilio WhatsApp + OSM/CARTO Tiles

Database: PostgreSQL + PostGIS (Neon)

Models: 6 Django models, 6 years of ward data

API Endpoints: 24 routes (public + JWT-authenticated)

A. Pitches

30-Second Elevator Pitch

UrbanIQ is a civic intelligence platform that turns 6 years of Mumbai ward data into live health scores, ML-powered risk forecasts, and AI-generated briefings - accessible via an interactive map dashboard and a WhatsApp complaint bot citizens can use without ever downloading an app.

2-Minute Detailed Pitch

UrbanIQ ingests 6 years of Praja Foundation data across 24 Mumbai wards and 18 complaint categories into a PostGIS-backed Django API. It computes a composite health score for every ward using a three-component sigmoid formula - weighing complaint volume, resolution speed, and civic engagement - then grades each ward Good, Moderate, or Poor.

On top of that, an XGBoost ML pipeline (classifier + quantile regressors + DBSCAN clustering) predicts next-year complaint volumes with confidence intervals, flags anomaly categories via z-score detection, and generates plain-English ward briefings with actionable recommendations - all served in under 500ms.

Citizens can submit geotagged complaints through a React + Leaflet dashboard or via a Twilio WhatsApp bot with a 6-step conversational flow that resolves ward by GPS or text. Councillors get a personalized portal with complaint maps, hotspot clusters, trend charts, and AI-generated action plans.

The entire stack - Django API, React SPA, ML models, static files - ships in a single Docker container on Render, with Unicorn handling requests and Celery queuing nightly model retraining and PDF reports.

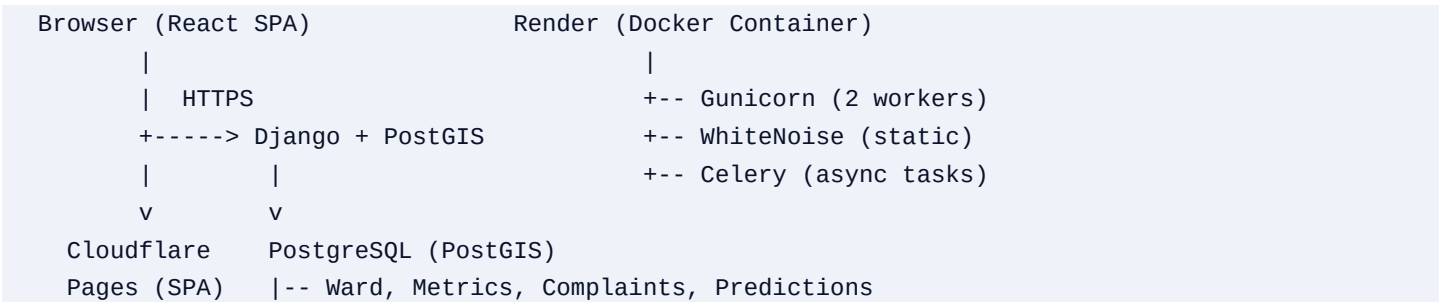
B. Table of Contents

1. Architecture
2. Backend
3. Frontend
4. Database Schema & Relationships
5. ML Pipeline
6. External Services & Third-Party Integrations
7. Important Features (Deep Dives)
8. Algorithms
9. Security
10. Performance & Scalability
11. Deployment
12. Interview Questions
13. Deep Dive Questions & Answers
14. Weaknesses & Technical Debt
15. Resume Talking Points
16. One-Page Cheat Sheet

1. Architecture

High-Level Architecture

Browser (React SPA) -> HTTPS -> Render (Docker) -> Gunicorn x2 -> Django + PostGIS -> PostgreSQL (PostGIS) | Redis (Celery) | Twilio API (WhatsApp)



Data Flow

1. React SPA loads -> useEffect fires parallel fetch() to API endpoints
2. Vite proxies /api/* to 127.0.0.1:8000 (dev); Cloudflare Functions proxy to Render (prod)
3. Django dispatches to @api_view function-based views
4. Views query ORM with select_related/prefetch_related; spatial queries via PostGIS ST_Contains
5. Data serialized inline as dicts
6. JWT auth: access tokens in localStorage, Bearer header
7. ML modules called synchronously from views

Design Patterns Used

- Service Layer: api/services/health_score.py - Separates business logic from views
- Strategy Pattern: twilio_views.py:_resolve_ward() - 4 fallback strategies for ward name resolution
- State Machine: twilio_views.py (6-step WhatsApp) - greeting->category->description->photo->location->confirm->done
- Signal/Slot: models.py @receiver(post_save, sender=User) - Auto-creates UserProfile on User creation
- Multi-Stage Build: Root Dockerfile (3 stages) - Node build stage keeps final image lean
- Repository Pattern: ml/utlis.py save/load model wrappers - Abstracts model persistence
- Proxy Pattern: Cloudflare [[path]].js - Hides backend origin from clients
- Module-level Caching: views.py (5-min TTL caches) - Avoids repeated CSV reads / DBSCAN re-runs
- Expanding Window CV: train.py - Time-series cross-validation respecting temporal order

Folder Structure

```
MumbaiUI/
+-- Dockerfile (multi-stage: Python + Node + final)
+-- backend/
|   +-- config/ (settings, urls, wsgi, celery)
```

```
|  +-- api/ (models, views, serializers, admin, auth, twilio)
|  |  +-- services/ (health_score, report_generator)
|  |  +-- management/commands/ (load_wards, load_metrics, ...)
|  +-- ml/ (train, predict, features, anomaly, briefing, ...)
|  |  +-- models/ (*.pkl files)
|  +-- data/ (CSV seed data)
|  +-- entrypoint.sh (startup script)
+-- frontend/
|  +-- src/ (components/, pages/, context/)
|  +-- functions/api/[[path]].js (Cloudflare proxy)
|  +-- vite.config.js
+-- README.md
```

2. Backend

Framework

Django 5.2 + DRF 3.17 + PostGIS. Function-based views with @api_view. No ViewSets.

Authentication (JWT via simplejwt)

Access token: 7 days lifetime

Refresh token: 30 days lifetime

Auth header: Bearer

Token blacklist: Enabled (logout blacklists refresh token)

Login supports: Username OR email (auth_views.py:28-51)

Authorization

- Default DRF permission: AllowAny
- IsAuthenticated on councillor dashboard and PDF download only
- Role checks manual: profile.role != 'councillor' -> 403

API Endpoints (24 Routes)

Public (no auth)

Endpoint	Method	Purpose
identify-ward/	GET	Reverse geocode lat/lng to ward
wards-geojson/	GET	All ward boundaries as GeoJSON
health-scores/	GET	Ward health scores (filterable)
complaints/	GET	Paginated complaints
complaints/submit/	POST	Create complaint
complaints/<pk>/	GET	Single complaint detail
councillors/	GET	Engagement scores
trends/	GET	5-year trends
hotspots/	GET	DBSCAN clusters
public/wards/	GET	Public ward data
public/summary/	GET	City-wide summary
public/config/	GET	WhatsApp config
auth/register/	POST	Register
auth/login/	POST	Login

Authenticated (JWT required)

Endpoint	Method	Purpose
councillor/dashboard/	GET	Full dashboard (councillor only)

reports/download/	GET	PDF report (councillor only)
auth/profile/	GET	Current user profile
auth/logout/	POST	Logout (blacklists token)
auth/login/refresh/	POST	Refresh access token
complaints/<pk>/status/	PATCH	Update complaint status
twilio/webhook/	POST	WhatsApp webhook (csrf_exempt)

Health Score Formula (health_score.py:34)

```
sigmoid(x, m, s) = 1 / (1 + exp(s * (x - m)))

complaint_score = sigmoid(per_capita, 6500, 0.0008)
resolution_score = sigmoid(avg_days, 40, 0.12)
deliberation_score = sigmoid(deliberations, 55, -0.10)

raw = 0.35*C + 0.35*R + 0.30*D
score = clamp(raw * 100, 0, 100)
label: >=70 Good, >=45 Moderate, else Poor
```

Engagement Score (views.py:260)

```
engagement = min(100, max(0, (per_capita_deliberations / 104) * 100))
```

Celery Tasks (api/tasks.py)

Task	Purpose
sync_complaints_to_portal_metrics()	Nightly Complaint -> PortalMetrics
generate_predictions()	2-year ML predictions for all wards
retrain_models()	Full ML pipeline retrain
generate_weekly_pdf_reports()	PDF reports for all wards

Note: Celery is configured but no worker runs in the Docker entrypoint.

3. Frontend

React 19 + Vite 8 + React Router 7. No TypeScript. Single App.css (~2500 lines). Dark theme.

Routing

Path	Component	Auth
/	LandingPage	No
/dashboard	Dashboard	No
/track	TrackComplaint	No
/login	Login	No
/signup	Signup	No
/admin-portal	AdminPortal	Admin
/councillor-portal	CouncillorPortal	Councillor
/public	PublicDashboard	No
/complaints-map	ComplaintsMap	No

State Management

- AuthContext.jsx: user, tokens, login/logout/register/fetchProfile
- Tokens in localStorage (access_token, refresh_token)
- Component-local useState for all UI state. No Redux/Zustand.

Maps (Leaflet + react-leaflet)

- 3 instances: LandingPage (all 24 wards), CouncillorPortal (ward), ComplaintsMap (all)
- Tile layers: CARTO dark (public), OpenStreetMap (councillor portal)
- GeoJSON ward boundaries with health-score color fill
- Custom L.divIcon markers, color-coded by 7 complaint categories
- DBSCAN hotspots as semi-transparent red circles

Charts (Recharts)

Chart	Usage
BarChart	Health scores, resolution days, deliberations
LineChart	Multi-year trends, ML predictions
PieChart (donut)	Complaint category distribution
RadialBarChart	Health score gauge (180-degree arc)

4. Database Schema & Relationships

PostgreSQL + PostGIS. Via `dj_database_url` in production (Neon on Render). GeoDjango.

6 Models

```
auth_user (Django built-in)
|
1:1 UserProfile (role, ward FK, phone)

Ward (ward_no, ward_name, boundary: MultiPolygonField, health_score)
|
1:N CivicMetrics (ward+year unique, 8 metric fields)
1:N Complaint (category, status, lat/lng, source, image)
1:N PortalMetrics (ward+year unique)
1:N WardPrediction (risk, complaints + intervals, health_score, recommendation, JSON features)
```

Constraints & Indexes

- `unique_together`: CivicMetrics(ward, year), PortalMetrics(ward, year)
- `on_delete=CASCADE` on all FKs except Ward in UserProfile (SET_NULL)
- PostGIS spatial index on Ward.boundary (MultiPolygonField)
- 14 migration files, no data migrations

5. ML Pipeline

Models

Model	Type	Purpose	Key Params
Risk Model	XGBClassifier	3-class risk (Low/Med/High)	n=150, depth=5, lr=0.1
Forecast Median	XGBRegressor q=0.5	Point prediction	quantile_alpha=0.5
Forecast Lower	XGBRegressor q=0.1	Lower bound (10%)	quantile_alpha=0.1
Forecast Upper	XGBRegressor q=0.9	Upper bound (90%)	quantile_alpha=0.9
Clustering	DBSCAN	Ward similarity groups	eps=1.5, min_samples=3

Features (features.py)

- 17 features: raw metrics + resolution_rate, escalation_rate
- Lags: complaints_lag1, complaints_lag2 (1yr and 2yr, grouped by ward)
- Rolling mean: 3-year complaints (min_periods=1)
- Growth rate: year-over-year pct_change (NaN filled 0)
- Training: y_complaints shifted -1 year (T features predict T+1)

Training & Saved Models

- 80/20 stratified split, StandardScaler, expanding window CV [2020..2024]

```
ml/models/  
risk_model.pkl      - XGBClassifier (3 classes)  
forecast_model.pkl  - XGBRegressor q=0.5 (median)  
forecast_lower.pkl  - XGBRegressor q=0.1 (lower bound)  
forecast_upper.pkl  - XGBRegressor q=0.9 (upper bound)  
cluster_model.pkl   - DBSCAN metadata  
scaler.pkl          - StandardScaler  
feature_columns.pkl - Expected feature column names
```

6. External Services

Service	Usage	Integration
Twilio	WhatsApp bot + status updates	twilio_views.py: REST + TwiML
OpenStreetMap	Map tiles (councillor portal)	Leaflet tile URL
CARTO	Dark map tiles (public dashboards)	Leaflet dark_all tiles
Neon (PostGIS)	Production database	DATABASE_URL env var
Redis	Celery broker (optional)	CELERY_BROKER_URL env var
Cloudflare Pages	SPA hosting + API proxy	functions/api/[[path]].js
WhiteNoise	Static file serving	Django middleware
Gunicorn	WSGI HTTP server	2 workers, 120s timeout

7. Important Features

Health Score

Files: health_score.py:34, features.py:16

3 sigmoid components -> weighted avg 35/35/30 -> clamp [0,100]. Sigmoid provides smooth S-curve normalization with tunable midpoint/steepness.

WhatsApp Bot

Files: twilio_views.py:137-325

6-step state machine (greeting->category->description->photo->location->confirm->done). Location: GPS (ST_Contains) or text (4-strategy resolver). In-memory state (Redis needed for scale).

Anomaly Detection

Files: anomaly.py:34-94

$z = (\text{latest} - \text{mean}) / \text{std}(\text{ddof}=1)$. Severity: $|z| \geq 3$ critical, ≥ 2 major, ≥ 1.5 minor. 18 categories x 13 years.

PDF Reports

Files: report_generator.py:33-188

fpdf2. Health score, metrics, categories, max 25 recent complaints. Trigger: Celery or /api/reports/download/.

GeoJSON Boundaries

Files: views.py:60-82

mumbai_wards.geojson -> PostGIS -> /api/wards-geojson/ -> Leaflet. ST_Contains spatial query.

Councillor Dashboard

Files: views.py:290-573 (283 lines)

~25 fields: health, metrics, complaints, ML predictions, anomaly, rankings, briefing. JWT + role check.

8. Algorithms

1. Sigmoid Health Score

```
sigmoid(x, m, s) = 1 / (1 + exp(s * (x - m)))  
complaint_score  = sigmoid(per_capita, 6500, 0.0008)  
resolution_score = sigmoid(avg_days, 40, 0.12)  
deliberation_score = sigmoid(deliberations, 55, -0.10)  
final = clamp((0.35*C + 0.35*R + 0.30*D) * 100, 0, 100)
```

2. Z-Score Anomaly Detection

```
z = (latest - mean) / std(ddof=1)  |z|>=3 critical, >=2 major, >=1.5 minor
```

3. DBSCAN Hotspot Clustering

```
DBSCAN(eps=0.005, min_samples=2) on complaint lat/lng  
Groups nearby complaints, center = mean lat/lng per cluster, cached 5-min TTL
```

4. Expanding Window CV

```
For test_year in [2020..2024]: train on 2019..test_year-1, evaluate on test_year
```

5. Category Scoring

```
score = 0.35*growth_norm + 0.35*esc_norm + 0.25*db_boost + 0.30*aff_boost + 0.15*ward_esc_rate
```

9. Security

JWT Authentication Flow

- Login -> POST /api/auth/login/ -> {access, refresh, user}
- Access token (7d) in localStorage -> Bearer header
- Refresh token (30d) -> /auth/login/refresh/ -> new access token
- Logout -> /auth/logout/ -> blacklists refresh token
- Frontend: on 401 auto-refresh; if fails, redirect to login

CSRF, CORS, Passwords

- CsrfViewMiddleware active; csrf_exempt on Twilio webhook
- CORS_ALLOW_ALL_ORIGINS=True in dev; DJANGO_CORS_ALLOWED_ORIGINS env var in prod
- Django set_password() (PBKDF2+SHA256); write_only=True on password fields
- No rate limiting, no Twilio signature validation (known weaknesses)

10. Performance & Scalability

Optimizations in Place

- `select_related('ward')`, `prefetch_related('metrics')` for N+1 prevention
- PostGIS spatial index on `Ward.boundary`
- `PageNumberPagination` on complaints (default 50, max 500)
- Module-level caches for escalation CSV and DBSCAN hotspots (5-min TTL)

Bottlenecks

- In-memory WhatsApp state - not horizontally scalable
- Synchronous ML in request cycle - blocks worker
- No frontend lazy loading - all routes eager
- No virtualization on complaint lists
- No connection pooling, no CDN, no read replicas

11. Deployment

Docker Multi-Stage Build

```
Stage 1: python:3.12-slim-bookworm + apt GDAL/GEOS + pip install
Stage 2: node:20-bookworm + npm install + npm run build
Stage 3: FROM stage1 + COPY --from=frontend dist/ + CMD entrypoint.sh
```

Startup Sequence (entrypoint.sh)

```
1. collectstatic 2. migrate
3. Guard: check api_ward table -> re-migrate if missing
4. load_wards 5. load_metrics 6. update_health_scores
7. Create councillor accounts 8. seed_complaints
9. exec gunicorn --workers 2 --timeout 120
```

Gunicorn

```
gunicorn config.wsgi:application --bind 0.0.0.0:$PORT --workers 2 --timeout 120 --log-level debug
```

- 2 workers (reduced from 4 for free tier memory)
- exec replaces shell for proper signal handling (SIGTERM -> Gunicorn)

12. Interview Questions

Beginner

Q1: What is UserProfile and how is it created?

Extends User with role, ward, phone. Auto-created via post_save signal on User (models.py:21-24).

Q2: How does the frontend know which API URL to call?

config.js checks window.__API_BASE__ or falls back to /api. Vite proxies to Django in dev. Cloudflare Functions to Render in prod.

Q3: What happens when a user submits a complaint?

Frontend POSTs FormData to /api/complaints/submit/. Backend resolves ward (name or ST_Contains). Creates Complaint with status=open. Returns 201.

Q4: How are static files served?

WhiteNoise middleware (CompressedManifestStaticFilesStorage). STATICFILES_DIRS includes frontend/dist/assets/. SPA catch-all serves index.html.

Q5: What does exec gunicorn do?

Replaces shell process with Gunicorn. Ensures SIGTERM reaches Gunicorn for graceful shutdown.

Intermediate

Q6: How does the health score work?

Three sigmoid components (35/35/30): complaint (per_capita, 6500, 0.0008), resolution (avg_days, 40, 0.12), deliberation (deliberations, 55, -0.10). Clamped [0,100].

Q7: How does WhatsApp bot handle state?

In-memory CONV_STORE dict keyed by phone number. Stores current step and partial data. Webhook reads, validates, transitions, stores back.

Q8: How is JWT auth implemented on FE?

AuthContext.jsx: stores tokens in localStorage, auto-refresh on 401, Bearer header on requests.

Q9: What ML models are used?

XGBClassifier (3-class risk), XGBRegressor x3 (quantile 0.1/0.5/0.9 for intervals), DBSCAN (clustering).

Q10: How does expanding window validation work?

For each year in [2020..2024]: train on prior data, build shifted features (T->T+1), evaluate. No future leakage.

Advanced

Q11: Why function-based views over ViewSets?

Each endpoint has unique behavior (DBSCAN, trend aggregation, 25-field dashboard). FBC makes contract explicit. Only ~20 endpoints, so boilerplate is minimal.

Q12: What are scalability bottlenecks?

In-memory CONV_STORE, module-level caches (per-process), synchronous ML, no connection pooling, monolithic Django process, no read replicas.

Q13: Explain WhatsApp location resolution.

GPS: Point(lng,lat,4326) -> ST_Contains on Ward boundaries. Text: 4-strategy matcher (exact -> strip 'ward' -> remove separators -> single letter).

Q14: How does iterative ML prediction work?

Standard: latest features -> next year. Iterative: N+1 prediction updates complaint_growth_rate for N+2 features. Error accumulates with each step.

Q15: How is the SPA served from Django?

TEMPLATES[0].DIRS = [frontend/dist/]. SPA catch-all: `re_path(r'^.*$', TemplateView(index.html))`. Assets at `/assets/` via `django.views.static.serve`.

Q16: Why csrf_exempt on Twilio webhook?

Twilio sends POST without CSRF token. Safe because webhook URL is registered in Twilio console with request signature validation.

13. Deep Dive Questions

Why XGBoost over LSTM?

Dataset is tiny (144 rows after features). XGBoost handles small tabular data, provides feature importance, trains in seconds. Quantile regression gives prediction intervals natively.

Walk through a complete request.

User clicks ward on Leaflet map -> onWardClick sets selectedWard -> WardDetailPanel renders. API: fetch(/api/wards-geojson/) -> Django -> Ward.objects.all() -> ST_AsGeoJSON -> JSON -> Leaflet renders health-colored polygons.

What if Twilio webhook fails?

WhatsApp complaints fail. Status notifications fail. In-memory state lost on restart. Web portal and all other endpoints remain unaffected.

How does councillor login work?

Login form -> AuthContext.login() -> POST /api/auth/login/ -> authenticate(username/password) with email fallback -> JWT tokens -> localStorage -> redirect to /councillor-portal -> role check -> Bearer auth on API calls.

Key tradeoffs?

In-memory state (simple but not durable). FBC (explicit but verbose). Synchronous ML (simple but blocks). Single container (easy but no scaling). 7-day JWT (fewer refreshes but longer vulnerability). Monolithic CSS (fast but 2500 lines).

14. Weaknesses & Technical Debt

Issue	Location	Fix
In-memory WhatsApp state	twilio_views.py	Redis-backed store
Module-level caching	views.py	Redis/Django cache
No TypeScript	All frontend	Migrate incrementally
Monolithic CSS	App.css (~2500 lines)	CSS modules
No DRF serializers for Complaint	views.py	Add ComplaintSerializer
Hard-coded ward list in JSX	Multiple files	Fetch from API
No rate limiting	auth_views.py	django-ratelimit
No Twilio signature validation	twilio_views.py	Twilio request validator
Synchronous ML	views.py	Celery task + polling
No lazy loading	App.jsx	React.lazy() + Suspense
No tests	Entire project	pytest + jest

15. Resume Talking Points

Technical Achievements

- Full-stack civic platform: Django REST API + React SPA + PostGIS + ML pipeline
- Sigmoid health score weighing 3 metrics with tunable parameters
- XGBoost ML pipeline with quantile regression for prediction intervals
- Twilio WhatsApp bot: 6-step state machine, dual-mode location resolution
- Multi-stage Docker build on Render (Gunicorn + WhiteNoise)
- 5 Django models, 14 migrations, PostGIS spatial extensions
- 24 REST API endpoints, JWT auth, role-based access control

Key Metrics

- 24 Mumbai wards, 6 years civic data, 8 metrics per ward/year
- 18 complaint categories, 13-year z-score anomaly detection
- XGBClassifier + 3 XGBRegressors + DBSCAN with expanding window CV
- 7 category-colored map markers, 3 map instances, 4 chart types
- 2 Gunicorn workers, 120s timeout, single Docker container

16. One-Page Cheat Sheet

Architecture

Browser (React SPA) -> Cloudflare Functions -> Render (Docker) -> Gunicorn -> Django -> PostGIS + Redis + Twilio

Tech Stack

BE: Django 5.2, DRF 3.17, PostGIS, Gunicorn, Celery 5.6

FE: React 19, Vite 8, Leaflet, Recharts 3

ML: XGBoost 2.1, scikit-learn, pandas, joblib

Infra: Render, Cloudflare Pages, Neon, Twilio

Database (6 Models)

Ward -> CivicMetrics | Complaint | PortalMetrics | WardPrediction | UserProfile -> auth_user

Auth Flow

Register/Login -> JWT (access 7d + refresh 30d) -> localStorage -> Bearer header -> simplejwt validates -> role check

Health Score

```
score = [sigmoid(per_capita, 6500, 0.0008)*0.35 + sigmoid(avg_days, 40, 0.12)*0.35 + sigmoid(deliberati
```

Key Decisions

- FBC over ViewSets: explicit control
- Sigmoid: smooth normalization, tunable
- JWT: stateless, horizontally scalable
- Single container: simple deployment

Deployment Flow

- Git push -> Render builds Dockerfile
- Startup: collectstatic -> migrate -> seed (wards/metrics/scores/accounts/complaints) -> gunicorn

Weaknesses to Acknowledge

- In-memory WhatsApp state, no TypeScript, no tests, no rate limiting, synchronous ML, no CDN